

SISTEMAS OPERATIVOS I



ADMINISTRADOR DE PROCESOS



Por: Ing. Edwin Calle Terrazas

Un **proceso** es una unidad de actividad que se caracteriza por la ejecución de una **secuencia de instrucciones**, un conjunto asociado de recursos del sistema , y un **estado actual**".

Otra definición de proceso, es : "Un **proceso** es un código que corre (run) independiente de otro código"

Se puede decir entonces que:

- *Un proceso es la ejecución de una secuencia de instrucciones.* Obviamente, porque un proceso es básicamente un algoritmo o programa, ejecutándose.
Recordemos que un algoritmo o programa es una secuencia de instrucciones o pasos.
- Un proceso tiene asociado un conjunto de recursos del sistema. Todo proceso necesita recursos para correr. El primer recurso importante que se le asocia es el espacio de RAM donde el proceso está alojado. El segundo recurso en importancia, es el quantum (ejecución de las instrucciones del proceso, por parte de la CPU).
- Un proceso puede estar en un estado. Los estados de un proceso se detallan en la siguiente diapositiva.

Running (*Corriendo*)

La CPU está ejecutando el código del proceso. Es decir, el proceso está haciendo uso del quantum.

Ready (*Listo o Preparado*)

El proceso no tiene impedimentos para correr, pero la CPU está corriendo el código de otro proceso. En otras palabras, el proceso no está haciendo uso del quantum, sino que está aguardando por él.

Wait (*Espera o bloqueado*)

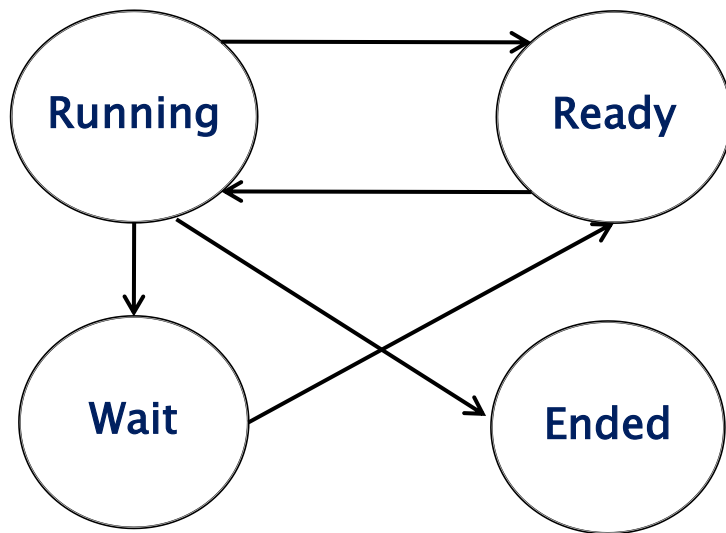
El proceso **tiene impedimentos** para correr: Está esperando a que uno de sus subprocesos finalice.

Ended (*Finalizado*)

El proceso ha terminado su ejecución. Cuando esto ocurre, el proceso es desalojado de la RAM (FreeMem) y todos los demás recursos que él utilizó le son devueltos al SO.

El **Grafo de Estados** de un proceso, es una representación gráfica de las transiciones (cambio de estado) que un proceso experimenta o podrá experimentar durante su permanencia en el sistema. Los vértices representan a los Estados y las aristas (flechas), las transiciones.

En la gráfica de la izquierda, se muestra el Grafo de Estados de cualquier proceso.



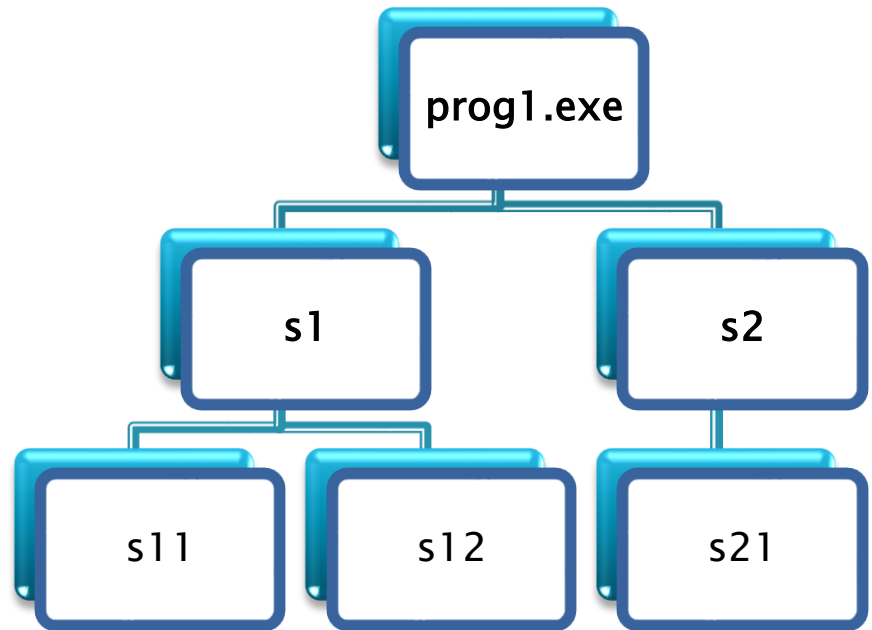
Nota. En el Grafo de Estados no se toma en cuenta a la orden Kill (terminar proceso) dada por el usuario.

Grafo de Estados General

- La transición **Running→Ready**, se da cuando el proceso deja de ser atendido por la CPU. Es decir, cuando el proceso acaba de agotar su quantum.
- La transición **Ready→Running**, es experimentada por el proceso cuando recupera el quantum (i.e. la CPU vuelve a ejecutar su código).
- El cambio de estado **Running→Wait**, lo hace el proceso, cuando genera un subproceso y éste último lo deja en espera
- Cuando está en Wait, el proceso es ignorado por la CPU porque es depositado en otra Estructura de Datos (ED), diferente a la de los procesos que están en Ready. Cuando el subproceso que lo mantiene en Wait finaliza, el SO traslada el proceso a la ED donde están los procesos en Ready. Por éste motivo, observamos: **Wait→Ready**.
- Cuando la CPU ejecuta la “última línea” del proceso (o sea el proceso aún está en running), el SO finaliza al proceso: **Running→Ended**.

Puesto que un subprocesso es también un proceso, éste a su vez puede generar subprocessos, generando así un árbol de (sub)procesos.

En la gráfica de la derecha, por ejemplo, se observa el árbol de procesos generado por el proceso padre **prog1.exe**. El proceso padre genera los subprocessos **s1** y **s2**. A su vez, **s1** genera los subprocessos **s11** y **s12**, mientras que **s2** genera el subprocesso **s21**.

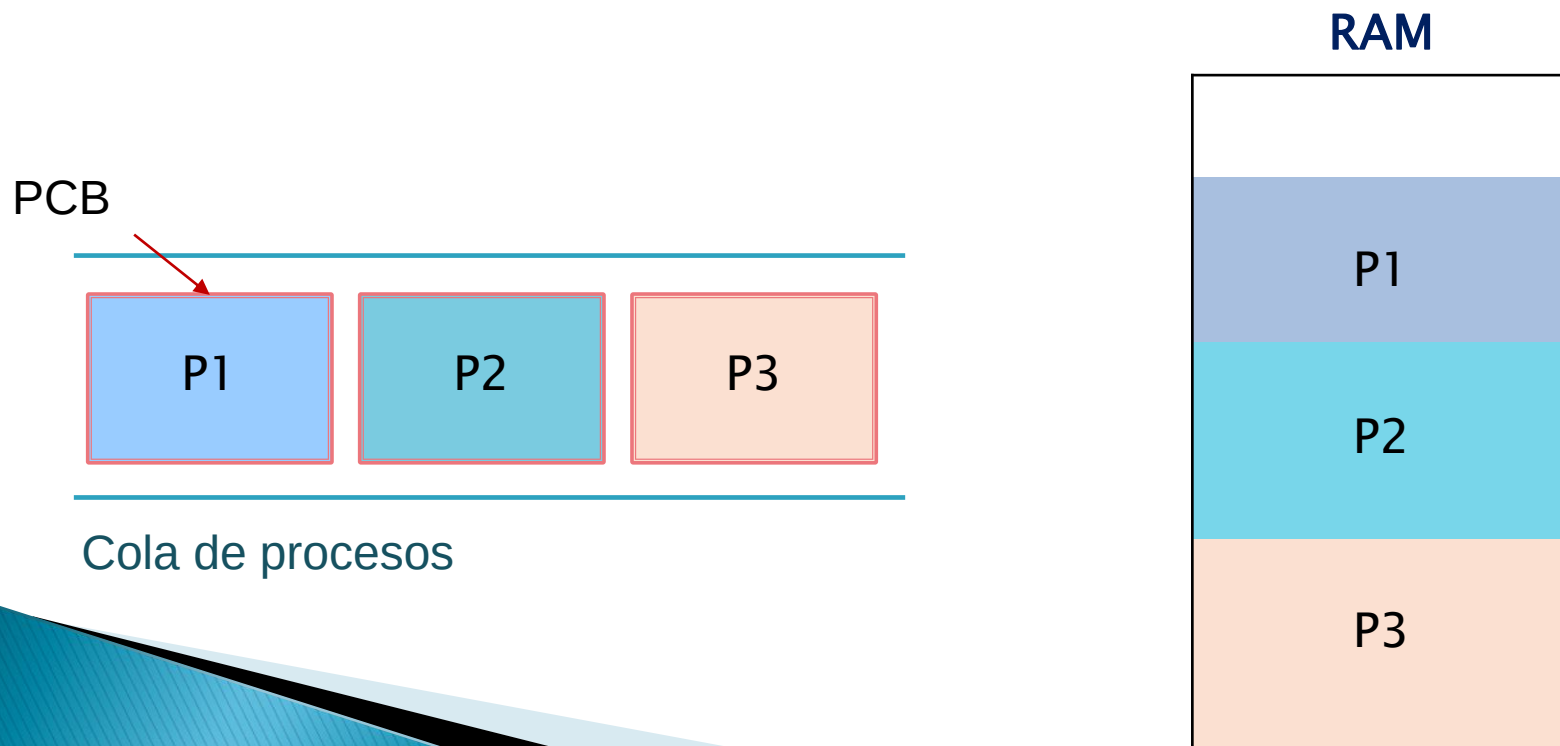


EL ADMINISTRADOR DE PROCESOS

- El problema de la multiprogramación radica en otorgar uno o más procesos a varios procesos que compiten por estar en modo “running”
- El mismo administrador de procesos es también un proceso.
- El administrador de procesos se encarga de implementar la capacidad multiproceso de un sistema operativo
- A grandes rasgos, lo que hace es gestionar qué procesos tienen derecho a usar la CPU y por cuanto tiempo

- A la unidad de tiempo mas pequeña que se puede asignar se denomina «**quantum**»
- *El quantum es la unidad de tiempo única e indivisible»*
- El quantum es una unidad **abstracta** de tiempo (no es absoluta, no se mide en 'ms')
- De ser necesario, controlamos la **cantidad de quantums entregados** a un proceso mediante **contadores**

Cada proceso que se encuentra en modo de ejecución, es anotado en un PCB (Process Control Block) así mismo, todos los PCB's se alinean en una cola de procesos, lo cual es administrada por el administrador de procesos

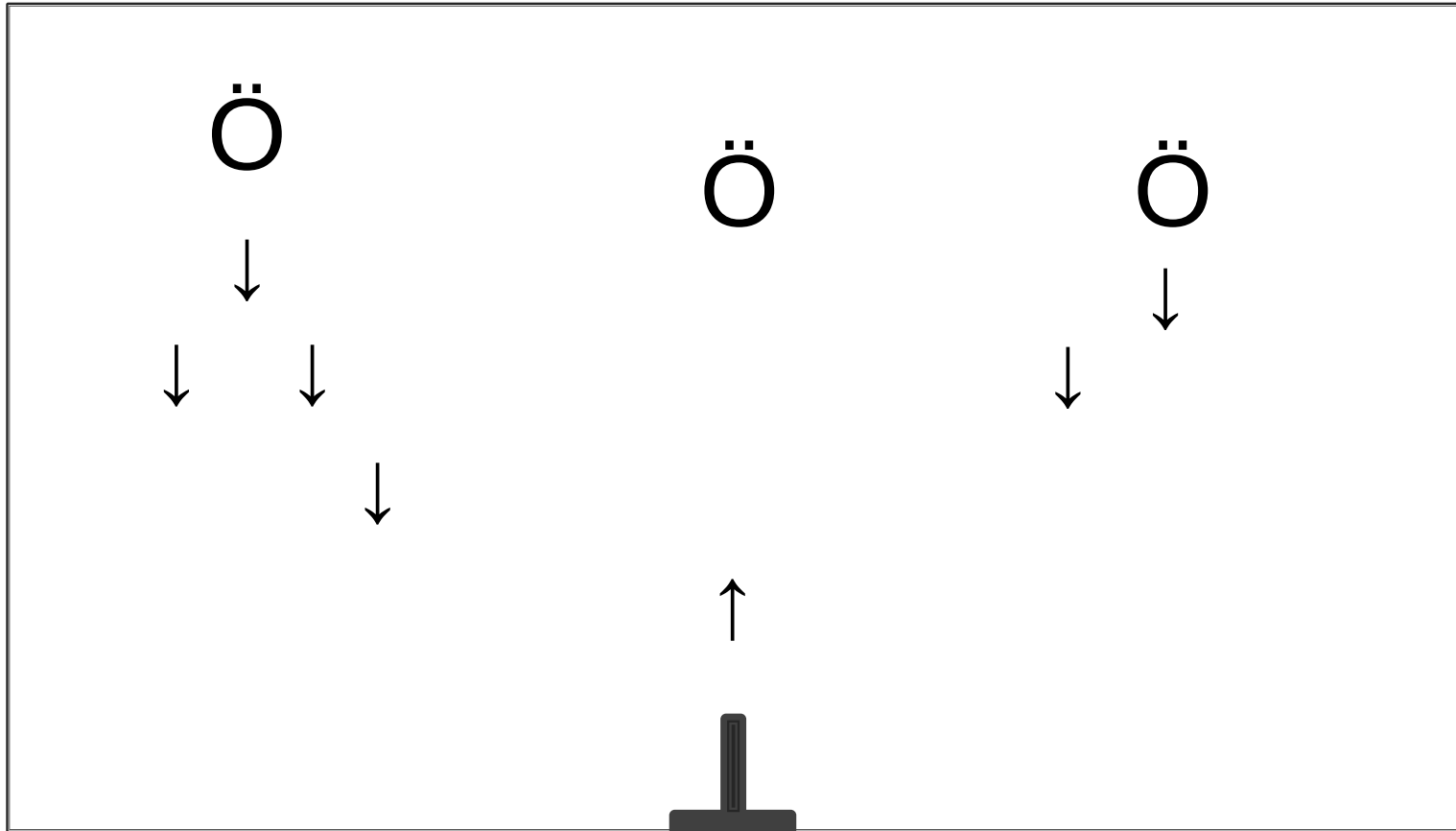


PROCESS CONTROL BLOCK (PCB)

Los PCBs son estructuras de datos que el sistema operativo usa para controlar a los procesos. Se componen de:

- PID (process ID)
- Nombre de imagen (Ej: programa.exe)
- Registros
- BlockRAM
- Otros (Ej: prioridad)

Se planteará un administrador de procesos para un juego dinámico



ALGORITMO DE PLANIFICACIÓN FIFO

- Los procesos pasan a la CPU en orden de llegada a la Cola de procesos listo
- Cada proceso va terminando su ejecución para darle paso al siguiente proceso
- Fácil de implementar, ya que se necesita tan solo mantener una lista de tareas activas. Es el algoritmo más sencillo; el primer proceso que solicita la CPU es el primero en recibirla
- Algoritmo no expulsivo



$$TE = TEJ - TLL$$

$$TR = TRaf + TRProAnt$$

$$TEProm = \sum TE / NP$$

$$TRProm = \sum TR / NP$$

Donde:

TE=Tiempo de espera

TEJ=Tiempo de ejecución

TLL= Tiempo de llegada

TEProm=Tiempo de espera promedio

TR=Tiempo de respuesta

TRaf=Ráfaga de CPU

TRProAnt=Tiempo de respuesta del proceso anterior

NP=Número de procesos

TRProm=Tiempo de respuesta promedio

TR=Tiempo que termina

Proceso	Tiempo llegada	Ráfaga de CPU
P1	0	10
P2	2	12
P3	4	5
P4	3	6
P5	1	24



$$TE(P1)=0-0=0$$

$$TE(P5)=10-1=9$$

$$TE(P2)=34-2=32$$

$$TE(P4)=46-3=43$$

$$TE(P3)=52-4=48$$

$$TE_{Prom}=(0+9+32+43+48)/5=26.4$$

$$TR(P1)=10+0=10$$

$$TR(P5)=24+10=34$$

$$TR(P2)=12+34=46$$

$$TR(P4)=6+46=52$$

$$TR(P3)=5+52=57$$

$$TR_{Prom}=(10+34+46+52+57)/5=39.8$$

Realizar lo siguiente;

a) Diagrama de Gant

b) Encontrar los tiempos de espera y los tiempos de respuesta de cada proceso,

c) Tiempo de espera promedio

d) Tiempo de respuesta promedio

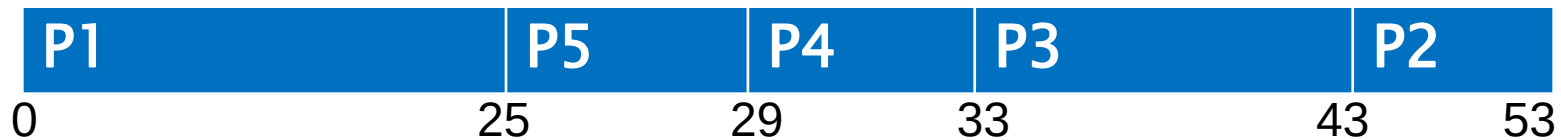
Proceso	Tiempo llegada	Ráfaga de CPU
P1	0	5
P2	2	7
P3	1	12
P4	3	3
P5	6	15
P6	4	4
p7	5	10

ALGORITMO DE PLANIFICACIÓN SJF (Shortest Job First) Primero el trabajo más corto

- Asociar a cada proceso el tiempo de ráfaga de CPU
- Selecciona el proceso con menor ráfaga de CPU
- En caso de empate aplicar FIFO
- Algoritmo no expulsivo



Proceso	Tiempo llegada	Ráfaga de CPU
P1	0	25
P2	4	10
P3	3	10
P4	2	4
P5	1	4



$$TE(P1)=0-0=0$$

$$TE(P5)=25-1=24$$

$$TE(P4)=29-2=27$$

$$TE(P3)=33-3=30$$

$$TE(P2)=43-4=39$$

$$TE_{Prom}=(0+24+27+30+39)/5=24$$

$$TR(P1)=0+25=25$$

$$TR(P5)=4+25=29$$

$$TR(P4)=4+29=33$$

$$TR(P3)=10+33=43$$

$$TR(P2)=10+43=53$$

$$TR_{Prom}=(25+29+33+43+53)/5=36.6$$

Proceso	Tiempo llegada	Ráfaga de CPU
P1	1	4
P2	1	3
P3	2	2
P4	1	1



Proceso	Tiempo llegada	Ráfaga de CPU
Word	0	8
Excel	2	6
PowerPoint	4	4
Calc	6	2



Realizar lo siguiente;

a) Diagrama de Gant

b) Encontrar los tiempos de espera y los tiempos de respuesta de cada proceso,

c) Tiempo de espera promedio

d) Tiempo de respuesta promedio

Proceso	Tiempo llegada	Ráfaga de CPU
P1	0	8
P2	2	15
P3	1	12
P4	3	9
P5	4	13

PLANIFICACIÓN ROUND ROBIN (RR)

Turno rotativo (Round Robin)

- La cola de procesos es circular (se puede implementar con una FIFO).
- El planificador la recorre y asigna un tiempo máximo de CPU (Q cuanto de tiempo) a cada proceso .

Un proceso puede abandonar la CPU

- Librementemente, si ráfaga de CPU $< Q$.
- Después de una interrupción, si ráfaga de CPU $> Q$.

Proviene del principio de turno rotativo, conocido en otros campos, donde cada persona toma una parte de un algo compartido en cantidades parejas.

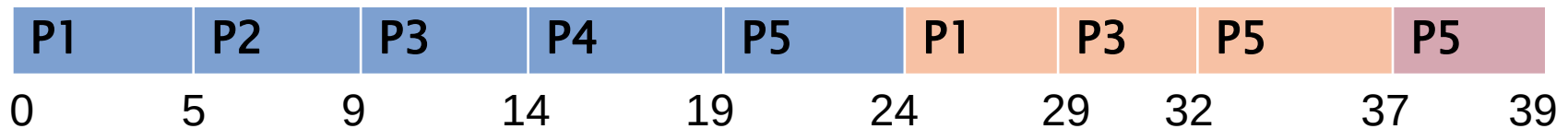


- Todos llegan en tiempo 0
- A cada proceso se le asigna un intervalo de tiempo llamado “quantum” y se representa con la letra q
- Cada proceso se ejecuta durante este quantum de tiempo
- Cuando un proceso recibe la CPU, puede ocurrir 2 cosas:
 1. Que el proceso tenga una ráfaga de CPU menor o igual que el quantum: en este caso, el proceso termina antes del quantum y planifica un nuevo proceso.
 2. Que el proceso tenga una ráfaga de CPU mayor que el quantum: en este caso el quantum para este proceso se agota, a continuación, es expulsado de la CPU dando paso a otro proceso y el proceso expulsado se colocará al final de la cola de procesos listos.

Proceso	Tiempo llegada	Ráfaga de CPU
P1	0	10
P2	1	4
P3	2	8
P4	3	5
P5	4	12

Todos llegan
en tiempo 0

$q=5$



$$TE(P1)=0+19=19$$

$$TE(P2)=5$$

$$TE(P3)=9+15=24$$

$$TE(P4)=14$$

$$TE(P5)=19+8=27$$

$$TE_{Prom}=(19+5+24+14+27)/5=17.8$$

$$TR(P1)=29$$

$$TR(P2)=9$$

$$TR(P3)=32$$

$$TR(P4)=19$$

$$TR(P5)=39$$

$$TR_{Prom}=(29+9+32+19+39)/5=25.6$$